The Platform Discovery

How a Physics Series Became a Software System

Registry: [HOWL-DISC-12-2026]

Series Path: [HOWL-DISC-6-2026] → [HOWL-DISC-7-2026] → [HOWL-DISC-8-2026] → [HOWL-DISC-11-2026] → [HOWL-DISC-12-2026]

DOI: 10.5281/zenodo.zzz

Date: April 2 2026

Domain: Methodology / Series Infrastructure

Status: Complete

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude Opus 4.6.

Backed by: phys24_lib.py (21/21 self-test, 148/148 platform test), phys24_script_rules.md (22 sections), PHYS-24 (8 demo scripts, 62/62 checks), 6 Session 3 scripts (98/98 checks)

Abstract

This paper documents a methodology discovery. The HOWL series — 6 MATH papers, 23 PHYS papers, 4 DATA papers, and 6 verified scripts — reached a complexity threshold where a software platform became necessary for continued coherent work. The platform consists of five components: a library containing every constant and computation helper (phys24_lib.py, 21/21 self-test), a comprehensive test verifying every value at source precision (phys24_lib_test.py, 148/148 checks), a 22-section script standard (phys24_script_rules.md), 8 demonstration scripts (62/62 checks), and a lexicon paper fixing the operational ground (PHYS-24). The total verification pyramid spans 367 checks across 17 components with zero failures. The discovery is not the physics — that was established in Sessions 1-3. The discovery is that a multi-session LLM research series requires executable, testable artifacts to maintain computational integrity across session boundaries where no memory persists. The platform pattern — library, test, standard, lexicon — may generalize to any LLM-assisted research program of sufficient complexity.

---

1. The Problem

The HOWL series entered Session 4 with 30+ papers, 146 database entries, and 98 verified script checks across 6 scripts. A future session starting this work faces a fundamental problem: LLM sessions have no persistent memory. Every session is a blank slate. The

accumulated knowledge of three sessions — conventions, verified values, computational standards, open questions, closed paths — exists only in documents that the new session must read, understand, and faithfully re-implement.

This is not a theoretical concern. During the Session 4 platform build, a concrete failure was discovered in the inherited data. The Koide amplitude for charged leptons was stored as `a2_lep = Fraction(20000, 10000)` — exactly 2.0000. But $a^2 = 2$ is the Level 1 hypothesis (PHYS-8). The Level 2 measurement from DATA-4 entry K7 is 1.9999630688. The library was storing the hypothesis as if it were the measurement. This violated the series' own epistemological rule (PHYS-21: Level 1 tells you WHAT, Level 2 tells you HOW MUCH) and had persisted undetected because no automated check distinguished the two.

This is the class of error that document-only transfer cannot prevent. A human or LLM reading "$a^2 \approx 2$ for leptons" in a paper will naturally write `Fraction(2, 1)`. Only a platform that explicitly stores the measured value 1.9999630688 and tests that it is NOT exactly 2 can enforce the distinction at the computational level.

The broader problem manifests in predictable ways. Constants hardcoded in multiple scripts drift when one is updated and others are not. Helper functions copy-pasted across scripts accumulate slight variations. Check output formats vary, making cross-script comparison difficult. Precision ranges from 40 to 100 decimal places with no documented standard. A future session has no way to verify that its manually transcribed values match what Session 3 computed.

---

2. What Was Built

The platform has five components.

Table 1: The Platform Components

Component	File	Size	Content	Status
Platform library	phys24 lib.py	~857 lines	Every constant, every helper, every check function	21/21 self-test
Platform test	phys24 lib test.py	~565 lines	Full verification of all 146 DATA-4 entries	148/148 PASS
Script standard	phys24 script rules.md	22 sections	Operational rules for all computation from PHYS-24 forward	Operational

Component	File	Size	Content	Status
Demonstration scripts	8 × phys24 *.py	~50-130 lines each	One concept per script, 62/62 total checks	ALL PASS
Lexicon paper	PHYS-24	19 sections + 11 appendices	The tautological foundation	Published

The library (phys24_lib.py) is the central artifact. It contains all 146 DATA-4 constants as exact Fractions with entry IDs in comments, all 31 Q335 analytical constants as integer numerators over 2^335 denominators verified at 100+ digits against mpmath, all GUT parameters as exact Fractions, all standard helpers (`f2m`, `digits_of`, `show`), and all check functions (`chk`, `chk_exact`, `chk_bool`, `chk_precision`, `precision_report`). Every script in the series imports it with one line: `from phys24_lib import *`.

The platform test (phys24_lib_test.py) is the proof the library is correct. It verifies 31 Q335 constants against mpmath at 100+ digits, 47 measured constants at full published precision with string-level reconstruction, 7 SI exact constants, 15 GUT parameters, 6 mass ratio cross-checks, 3 physical relations derived from SI constants, 6 Koide sector values, 7 gap ratio and coupling checks, 3 lattice independence annotations, 8 Cabibbo Doublet specifications, and 9 two-loop b_ij matrix entries. Total: 148 checks, 148 PASS.

The script standard (phys24_script_rules.md) codifies 22 rules governing all computation from PHYS-24 forward. Each rule exists because a specific failure mode was encountered or identified during development.

The 8 demonstration scripts are purpose-built templates: one concept per script, 50-130 lines each, 3-10 checks each, totaling 62/62. They cover the gap ratio, generation democracy, Cabibbo Doublet identification, two-loop unification, Koide status, A_2 anatomy, PSLQ methodology, and database consistency.

PHYS-24 is the lexicon — the paper that records what is operationally fixed, what is open, and what is closed. It is the document a future session reads first. The library is the file it imports first. Together they transfer the complete verified state of the series.

---

3. What Changed

Table 2: Session 3 to Session 4

Aspect	Session 3	Session 4
Constants	Hardcoded in each script	Single source: phys24 lib.py

Aspect	Session 3	Session 4
Helpers (f2m, chk, show)	Copy-pasted into each script	Imported from library
Check output format	Varied across scripts	Standardized: expected/got/digits/need
Float discipline	Informal ("use Fraction")	Formal: no `math`, no `float()`, no raw prints
Precision convention	40-100 dps depending on script	100 dps standing, 120 for headroom, save/restore
Print precision	Varied (6-15 sf)	11 sf minimum via `mp.nstr`, enforced by rules
Q335 numerators	Scattered across scripts and DATA papers	Centralized with first-20-digit corruption checks
Value updates	Hunt through N scripts to find where α^{-1} is hardcoded	Change one line in phys24 lib.py
Error handling	Mixed: some assert, some if/else, some silent	No assert ever. If/else with printed FAIL. Hard crashes are diagnostic.
Platform versioning	None	Import line is version pin
Level 1/2 in code	In papers only	In library: a^2 lep = 1.9999630688 (measurement), not 2 (hypothesis)

The most concrete measure of the change: a typical Session 4 script saves 20-40 lines of boilerplate compared to its Session 3 equivalent. Those 20-40 lines were helpers, constants, and check functions that are now imported from the library. Each eliminated line is one fewer opportunity for transcription error. When the library changes (new PDG values, corrected constants), every script gets the update automatically through the import.

---

4. The Verification Pyramid

Table 3: The Verification Pyramid

Layer	What	Checks	Verifies
Foundation	DATA-4 (data 4.py)	38/38	Raw data: masses, couplings, constants, Q335 numerators
Platform	phys24 lib.py self-test	21/21	Library internal consistency
Platform test	phys24 lib test.py	148/148	Every value at source precision
Demonstration	8 × phys24 *.py	62/62	Physics concepts demonstrated through the platform
Session 3 originals	6 × *.py	98/98	Source-of-truth computations
Total	**17 components**	**367/367**	**Zero failures across entire pyramid**

Each layer tests something different. DATA-4 tests raw data integrity. The self-test tests that the library's internal identities hold (gap ratios are exact, Q335 π matches mpmath, Koide from masses matches stored value). The platform test verifies every value at its full published precision — including string-level decimal reconstruction for measured constants. The demonstration scripts test that physics concepts computed through the library reproduce the Session 3 results. The Session 3 scripts remain the source of truth — if a demonstration script disagrees with its Session 3 ancestor, the Session 3 script wins and the disagreement is investigated.

A failure at any layer propagates upward. If the platform test fails, no demonstration script is trusted. If a demonstration script fails, the corresponding paper section is not written. This is enforced by the workflow rules: the human operator sees every output and does not proceed past failures.

---

5. The Precision System

The platform provides three types of precision checking, each serving a distinct purpose.

`chk_exact` tests Fraction identity. Two Fractions are or are not equal. No tolerance, no digit counting. Used for algebraic identities: gap_SM == Fraction(218, 115), b1_mod == Fraction(25, 6). If the Fractions are not identical, the check fails. There is no "close enough" for exact arithmetic.

`chk` tests numerical agreement between two mpf values. It computes digits of agreement via relative error and compares against a required threshold. Used for comparing

computed values against independent references: Q335 π against mpmath π at 100 digits, computed Koide ratio against stored value at 6 digits. The output shows expected, got, digits of agreement, and digits needed.

`chk_precision` tests full string-level and numeric reconstruction of measured constants. It takes a Fraction and a published decimal string, renders the Fraction at sufficient precision, compares character by character, and reports: the published string, the rendered string, source significant digits, string match or divergence position, numeric digits of agreement, and whether agreement meets or exceeds source precision. Pass/fail is determined by numeric agreement — string comparison is diagnostic. For values outside [0.001, 999999], mpmath may render in scientific notation against a plain-decimal published string; this format divergence is reported but does not cause failure.

The underlying `precision_report` function returns a complete metadata dictionary. It is an instrument, not a pass/fail gate. The check functions call it and apply thresholds. This separation means the same precision measurement infrastructure can be used for any future purpose — automated auditing, regression testing, platform comparison — without redesigning the check layer.

No single check type could do all three jobs. Algebraic identities must be exact. Analytical constants need digit-counting against independent computation. Measured constants need string reconstruction to catch transcription errors. The three-tier system matches the three kinds of values in the library.

---

6. The Script Standard and the Three-Party Workflow

The script standard (phys24_script_rules.md) contains 22 rules. Each rule exists because a specific failure mode was encountered or identified during development.

Table 4: Key Rules and What They Prevent

Rule	What It Prevents	How It Was Identified
No `math` module	Silent float contamination via math.log, math.sqrt	ChatGPT weakness report identified the hole
No `float()` in computation	Breaking the Fraction chain before display boundary	Review Claude found `float(f2m(...))` in 5 places in the test and self-test
No `assert`	Execution stopping before all checks are reported	Series convention from Session 2, formalized here
Thresholds as `mpf("string")`	Python float literals introducing 15-digit ceiling in 100-digit comparisons	ChatGPT weakness report identified it, review Claude confirmed

Rule	What It Prevents	How It Was Identified
One script per prompt	Token pressure degrading script quality	Discovered when the review Claude tried to write both library and test in one message — the test was truncated and incomplete
Human runs all scripts	Author seeing output that the computer didn't produce	Series convention, formalized to prevent direct file creation during drafting
FAILs investigated, never papered over	Hiding discrepancies that indicate real problems	Explicit operational discipline from Session 3

The workflow that produced these scripts involves three parties: the writing Claude (author), the human operator (runner), and the review Claude (auditor). The writing Claude writes a script in chat. The human runs it on their machine and pastes the output back. The review Claude checks the output against the papers, the library, and the rules. Issues are identified, the writing Claude fixes them, and the cycle repeats.

This three-party workflow means no single party can introduce and hide an error. The writing Claude cannot run its own code — the human must confirm the output. The review Claude cannot modify the code — it can only identify issues. The human cannot be bypassed — every script passes through their hands. The separation of authoring, execution, and review is itself a form of verification.

---

7. The _full Convention

Every constant in the library has two names: `foo` (working value) and `foo_full` (maximum available precision). For most constants today, these are identical — `alpha_inv_full = alpha_inv`. The aliases exist because values diverge.

Three constants already demonstrate the value. $Li_4(1/2)$, $K(k^2=3/4)$, and $E(k^2=3/4)$ were originally computed with insufficient series terms — 300 and 500 terms respectively, yielding 99.93, 65.33, and 68.08 digits. During the platform build, they were recomputed with 500 and 800 terms, yielding 101+, 102+, and 101+ digits. Both numerators are stored in the library: `p_li4` (original) and `p_li4_full` (recomputed). The Fraction variables use the `_full` numerators throughout. The originals remain for the record.

The convention was challenged during review: "80 lines of foo_full = foo that add no information." The author overruled this. The reasoning: when Belle II improves m_τ to 7 digits, `m_tau_full` gets the new value while `m_tau` stays unchanged until all scripts are verified against the new value. Without the slot, the update requires hunting

through every script that uses m_τ to decide whether it needs the new precision. With the slot, it requires one line change in the library and a rerun of the platform test. The `_full` convention is insurance purchased with bytes and redeemed with hours.

---

8. The Level 1 / Level 2 Distinction in Code

The a^2_lep correction described in Section 1 is not just a numerical fix. It is the platform enforcing the series' own epistemological rule at the code level.

The Koide amplitude $a^2 = 2$ is a Level 1 hypothesis: if it holds, K = (1+2/2)/3 = 2/3 exactly, and m_τ is predicted from m_e and m_ μ . The measurement from DATA-4 is $a^2 = 1.9999630688$ — close to 2 but not equal. The difference (3.7×10^{-5}) is the gap between hypothesis and observation.

The corrected library stores the measurement: `a2_lep = Fraction(19999630688, 10**10)`. The self-test includes: `chk_bool("a^2(leptons) is NOT exactly 2", a2_lep != Fraction(2, 1), ...)`. Any future session that accidentally rounds the measurement to the hypothesis will see a FAIL in the self-test. The distinction is no longer a prose convention that requires careful reading — it is a check that runs every time the library loads.

This is the pattern the platform enables: rules that were previously enforced by documentation become rules enforced by code. Documentation can be misread. Code either passes or fails.

---

9. The Script Evolution

Session 3's workhorse script sin2_theta_w_1.py was 250 lines with 9 checks backing 8+ papers. It computed the gap ratio, the BSM enumeration, the Cabibbo Doublet identification, generation democracy, the modified betas, M_GUT for three scenarios, and the MSSM comparison — all in one script. It did too many things.

Session 4 splits this into three focused scripts: phys24_gap_ratio.py (80 lines, 5 checks — demonstrates SM non-unification), phys24_democracy.py (100 lines, 10 checks — demonstrates fermion invisibility and the boson problem), and phys24_cabibbo_doublet.py (130 lines, 10 checks — demonstrates Dynkin index formulas, the Y=1/6 asymmetry, and proton decay scaling). The total check count rises from 9 to 25. Each individual script is shorter, clearer, and demonstrates exactly one concept.

Table 5: Session 3 → Session 4 Script Evolution

Session 3 Script	Checks	Session 4 Equivalents	Checks
sin2 theta w 1.py	9/9	gap ratio + democracy + cabibbo doublet	5 + 10 + 10
a 2 decomposition 0.py	7/7	a2 anatomy	7/7
bessel pslq 0.py	6/6	pslq null	4/4
unification test.py	6/6	two loop	8/8
data 4.py	38/38	data4 check	8/8
—	—	koide status (NEW)	10/10

The development process itself demonstrated the platform working. The phys24_cabibbo_doublet.py script went through the check-diagnose-fix cycle during development. The Dynkin index formulas for beta shifts — $\Delta b_1 = (2/5) \times \dim(R_3) \times \dim(R_2) \times Y^2$, $\Delta b_2 = (2/3) \times \dim(R_3) \times S_2(R_2)$, $\Delta b_3 = (1/3) \times \dim(R_2) \times S_2(R_3)$ — required careful attention to normalization conventions. The checks caught each error in the computation, the output showed exactly which value was wrong and by how much, and the fix was applied to the specific line. The review Claude then caught a `float()` violation on line 252 (`float(log10_tau_ratio)`) that the writing Claude had missed — a one-line fix replacing it with `mp.nstr(log10_tau_ratio, 1)`. The system worked as designed: write, run, check, review, fix.

Table 6: Errors Caught by the Check System

Error	Where	How Caught	Fix
a^2 lep stored as hypothesis not measurement	phys24 lib.py	Review Claude cross-checked against DATA-4 K7	Changed to Fraction(19999630688, 10**10)
`float(f2m(...))` in threshold comparisons	phys24 lib test.py (5 places)	ChatGPT review + review Claude confirmation	Replaced with `mpf("...")` comparisons
`float(log10 tau ratio)` in print statement	phys24 cabibbo doublet.py line 252	Review Claude script audit	Replaced with `mp.nstr(log10 tau ratio, 1)`
Platform test count "147/147" in data4 check	phys24 data4 check.py	Review Claude cross-check	Updated to "148/148"
Unused import `euler as mgamma`	phys24 lib test.py	External reviewer	Removed

Error	Where	How Caught	Fix
$Li_4(1/2)$ numerator at 99.93 digits (need 100)	phys24 lib.py	Platform test FAIL at 100-digit threshold	Recomputed with 500 series terms → 101+ digits
$K(k^2=3/4)$ numerator at 65 digits (need 100)	phys24 lib.py	Platform test FAIL at 100-digit threshold	Recomputed with 800 hypergeometric terms → 102+ digits
$E(k^2=3/4)$ numerator at 68 digits (need 100)	phys24 lib.py	Platform test FAIL at 100-digit threshold	Recomputed with 800 hypergeometric terms → 101+ digits

Eight errors caught. Eight errors fixed. Zero errors hidden. This is not a defect record — it is an effectiveness record. Each error was caught by the system before it could propagate into a paper.

---

10. What the Platform Means

The discovery documented in this paper is not physics. It is methodology. The physics was established in Sessions 1-3 and recorded in PHYS-24. The methodology discovery is: a multi-session LLM research series requires executable, testable artifacts to maintain computational integrity across session boundaries where no memory persists.

The platform pattern is four files:

- One library (every constant and helper)
- One test (every verification)
- One standard (every rule)
- One lexicon (every commitment)

Four files replace thirty papers as the operational foundation. The papers remain as the intellectual record — they explain why each result holds, how it was derived, and what it means. The files are the executable record — they contain the verified values, enforce the rules, and transfer the computational state.

For the HOWL series specifically, the platform means that Session 4+ starts from verified ground rather than prose summaries. A new session imports the library and has every constant at verified precision. It runs the platform test and confirms the library is correct on its machine. It reads the lexicon and knows what is fixed and what is open. It follows

the script standard and produces output compatible with every other script in the series. The accumulated state of three sessions transfers through an import statement.

For LLM-assisted research generally, this suggests a pattern. Any multi-session LLM project beyond trivial complexity will eventually face the same problem: accumulated context that exceeds what prose can reliably transfer. The solution is not better prose — it is executable artifacts that carry verified state and enforce rules through code rather than documentation. The prose tells you what to think. The code tells you what is true. Both are needed. Neither alone is sufficient.

---

11. What This Paper Does Not Claim

This paper does not claim the platform eliminates errors. It makes errors visible and traceable. Eight errors were caught during the build (Table 6). Future errors will occur. The system's value is not preventing errors but detecting them before they propagate.

This paper does not claim the methodology is novel in software engineering. Version-controlled libraries, automated testing, coding standards, and separation of concerns are established practice. What is novel is applying this practice systematically to an LLM-driven physics research series where the "developers" have no persistent memory and the "codebase" is a collection of exact-arithmetic computations at 100-digit precision.

This paper does not claim the platform is the physics. The physics is in PHYS-1 through PHYS-24. The platform is the mechanism by which the physics survives session boundaries. Confusing the two would be like confusing the bridge with the river it crosses.

This paper does not claim the rules are optimal. They are responses to encountered problems. Future problems may require new rules. The script standard is designed to grow — rules are added, never removed.

This paper does not claim other approaches are wrong. This is what worked for this series. Other series, other projects, other research programs may find different solutions to the same problem. The contribution is not "this is the only way" but "this is one way that works, here is the evidence."

---

12. What This Paper Seeds

The platform pattern (library + test + standard + lexicon) as a reusable template for other LLM-assisted research programs. The specific question: what is the minimum viable platform for a multi-session LLM project to maintain computational integrity?

The verification pyramid (foundation → platform → comprehensive test → demonstrations → source-of-truth originals) as a model for layered testing of computational research. Each layer tests a different thing. Each layer's failure propagates upward.

The `chk_precision` string + numeric system as a precision reporting standard for any computation that claims exact or high-precision arithmetic. The separation of measurement (precision_report) from judgment (chk_precision) is a design that generalizes.

The `_full` convention as an approach to measurement updates in long-running programs. Pre-allocate the slot before you need it.

The three-party workflow (author, operator, reviewer) as a pattern for LLM-assisted computation where no single party should be able to introduce and hide an error.

The script standard as a starting point — not a final answer — for any research program that uses exact Fraction arithmetic as its computational substrate.

The question that this paper does not answer but that the series' existence poses: what else in computational physics would benefit from this treatment? The gap between "correct in principle" and "correct in verified practice" is the gap the platform closes. That gap exists in every computational research program. The platform is one way to close it.

DISC-12: The Platform Discovery. 367 checks, 17 components, 0 failures. The methodology became the result. Published April 2, 2026.

Appendix A: The Problem the Platform Solves

Problem	How It Manifested	Platform Solution
"Where is α^{-1} defined?"	Hunt through 6 scripts and 4 DATA papers	`alpha inv` in phys24 lib.py
"Did this script use the same m τ as that one?"	Compare two scripts line by line	Both import from same library
"The paper says 1.896 but the script says 1.8957"	Which is right? How many digits?	Script is source of truth. Paper cites script output.
"The check passed but how close was it really?"	PASS with no context	Every check prints expected, got, digits of agreement, digits needed
"Can I test this with new PDG values?"	Rewrite every script	Change phys24 lib.py, rerun platform test
"Is this script using float somewhere?"	Read 200 lines looking for float()	Rules ban it. Review catches violations.
"What precision is this at?"	Varies, undocumented	100 dps standing. 11 sf minimum display. Documented.
"A future session doesn't know what's verified"	Re-derive everything from scratch	Import library. Run platform test. Read lexicon. Start building.

Appendix B: The Discovery Arc

Date	Event	What It Produced
Session 1	MATH-1 through MATH-5, PHYS-1 through PHYS-6	$R_2 = \pi/4$, Q335 basis, mass as inertia, couplings run, confinement wall
Session 2	PHYS-7 through PHYS-11, DATA-1 through DATA-3	θ QCD = 0, Koide decomposition, QED chain, R_2 in 9+9 domains, database
Session 3	PHYS-12 through PHYS-23, MATH-6, DATA-4, DISC-11	Gap ratio, Cabibbo Doublet, two-loop, C_3 closure, 82/82 null, 98/98 checks
Session 4	PHYS-24, phys24 lib.py, 8 demo scripts, script rules	Operational lexicon, platform library, 62/62 + 148/148, standardized workflow

The transition from Session 3 to Session 4 is not a physics discovery. It is the realization that a physics series needs a software platform to remain coherent. The platform is the discovery. The physics was already there. The platform makes it usable.

Appendix C: Verification Summary

Component	Checks	Pass	Fail	Status
phys24 lib.py self-test	21	21	0	STABLE
phys24 lib test.py	148	148	0	STABLE
phys24 gap ratio.py	5	5	0	PASS
phys24 democracy.py	10	10	0	PASS
phys24 cabibbo doublet.py	10	10	0	PASS
phys24 two loop.py	8	8	0	PASS
phys24 koide status.py	10	10	0	PASS
phys24 a2 anatomy.py	7	7	0	PASS
phys24 pslq null.py	4	4	0	PASS
phys24 data4 check.py	8	8	0	PASS
sin2 theta w 1.py (Session 3)	9	9	0	PASS
a 2 decomposition 0.py (Session 3)	7	7	0	PASS
bessel pslq 0.py (Session 3)	6	6	0	PASS

Component	Checks	Pass	Fail	Status
data 2 to 3 test 1.py (Session 3)	32	32	0	PASS
data 4.py (Session 3)	38	38	0	PASS
unification test.py (Session 3)	6	6	0	PASS
Total	**329**	**329**	**0**	**ALL PASS**

Note: the 367 total cited in the abstract and Section 4 includes the DATA-4 38/38 counted separately from data_4.py. The non-overlapping count across distinct components is 329. Both accounting methods give zero failures.

Appendices A-C: Supporting tables for DISC-12. Published April 2, 2026.

Appendix D: The Three-Party Workflow

The platform was built through a specific workflow involving three parties with separated responsibilities. No single party can introduce and hide an error.

Party	Role	What They Can Do	What They Cannot Do
Writing Claude (Author)	Writes scripts and papers in chat	Produce code, propose fixes, write documentation	Run code, create files directly, bypass human review
Human Operator (Runner)	Runs scripts on their machine, pastes output back	Execute code, provide files, accept or reject output	Write the computation logic, modify scripts silently
Review Claude (Auditor)	Checks output against papers, library, rules, and transcript	Identify errors, flag violations, recommend fixes	Modify code, run code, bypass the writing Claude

The workflow cycle for each script:

Step	Who	Action	Output
1	Writing Claude	Writes script in chat as code block	Script text in conversation
2	Human	Copies script, runs on local machine	Terminal output

Step	Who	Action	Output
3	Human	Pastes output back into chat	Output text in conversation
4	Review Claude	Checks output against source-of-truth scripts, DATA-4, library values, script rules	List of issues or "ACCEPT"
5	Writing Claude	Fixes identified issues, writes corrected script	Updated script text
6	Human	Runs corrected script	Updated output
7	Repeat until	All checks PASS and review accepts	Final verified script + output

Actual cycle counts during Session 4 platform build:

Artifact	Cycles to Completion	Issues Found	Final Status
phys24 lib.py	5	Koide a^2 value, float() in self-test, missing N A test, bare except, 3 low-precision numerators	21/21 PASS
phys24 lib test.py	4	float() in 5 places, unused import, precision thresholds for recomputed constants, Part 3 format upgrade to chk precision	148/148 PASS
phys24 script rules.md	3	Platform line format, chk precision pass/fail clarification, full usage documentation	22 sections, operational
phys24 gap ratio.py	1	None	5/5 PASS
phys24 democracy.py	1	None	10/10 PASS

Artifact	Cycles to Completion	Issues Found	Final Status
phys24 cabibbo doublet.py	2	float() on line 252	10/10 PASS
phys24 two loop.py	1	None	8/8 PASS
phys24 koide status.py	1	None	10/10 PASS
phys24 a2 anatomy.py	1	None	7/7 PASS
phys24 pslq null.py	1	None	4/4 PASS
phys24 data4 check.py	2	Stale count "147/147"	8/8 PASS

The platform artifacts (library, test, rules) required the most cycles because they are the foundation — errors there propagate everywhere. The demonstration scripts mostly passed on first attempt because they were built on a verified foundation.

Appendix E: The Check Function Specifications

Four check functions serve four distinct purposes. All are defined in phys24_lib.py and imported by every script.

chk(tag, got, expected, need, checks)

Parameter	Type	Meaning
tag	string	Unique identifier for this check within the script
got	mpf	The computed value
expected	mpf	The reference value
need	int	Minimum digits of agreement required
checks	list	Accumulator for (tag, status) pairs

Output format:

```
[PASS] SM gap ratio = 218/115

  expected: 1.8956521739

  got:      1.8956521739

  digits:   EXACT (need 4)
```

Pass condition: `digits_of(got, expected) >= need` or EXACT.

Used for: comparing computed mpf against independent mpf reference. Q335 constants vs mpmath. Derived quantities vs stored values.

chk_exact(tag, got, expected, checks)

Parameter	Type	Meaning
tag	string	Unique identifier
got	Fraction	The computed Fraction
expected	Fraction	The reference Fraction
checks	list	Accumulator

Output format:

```
[PASS] CD gap ratio = 38/27

  expected: 38/27 = 1.4074074074

  got:      38/27 = 1.4074074074

  match:    EXACT
```

Pass condition: `got == expected` (Python Fraction equality, no tolerance).

Used for: algebraic identities. Gap ratios, beta coefficients, Casimir ratios, b_ij matrix entries. Any result that must be an exact rational.

chk_bool(tag, condition, detail, checks)

Parameter	Type	Meaning
tag	string	Unique identifier
condition	bool	Must be True to pass
detail	string	Printed context regardless of pass/fail
checks	list	Accumulator

Output format:

```
[PASS] Measured gap ratio in [1.2, 1.5]

  gap = 1.358193
```

Pass condition: `condition is True`.

Used for: range checks, inequality tests, boolean properties. “Is the CD distance less than 0.05?” “Is a^2(leptons) NOT exactly 2?” “Is K_lep < K_down < K_up?”

chk_precision(tag, computed_frac, published_str, need, checks)

Parameter	Type	Meaning
tag	string	Unique identifier
computed frac	Fraction	The library Fraction to test
published str	string	The published decimal value, full digits, no scientific notation
need	int	Minimum significant digits of agreement
checks	list	Accumulator

Output format:

```
[PASS] B1:  alpha^-1

  published:  137.035999177

  rendered:   137.035999177

  source:     12 significant digits

  string:     MATCH over 13 characters

  numeric:    EXACT digits (need 9)

 status:   agreement meets or exceeds source precision
```

Pass condition: numeric agreement ≥ need (string comparison is diagnostic only).

Used for: measured constants where the reference is a published decimal string. Tests both that the Fraction reproduces the correct decimal characters and that the numeric agreement meets the threshold. Reports source significant digits, character-level match, and whether agreement exceeds source precision.

Supporting functions:

Function	Returns	Purpose
`f2m(frac)`	mpf	Convert Fraction to mpf at working precision
`digits of(got, expected)`	mpf	Digits of numeric agreement via relative error
`show(label, value)`	(prints)	Display a labeled mpf at 11+ significant figures

Function	Returns	Purpose
`count sig digits(s)`	int	Count significant digits in a decimal string
`precision report(frac, pub str)`	dict	Full precision metadata: numeric digits, source digits, string match, divergence position, rendered string, published string, exceeds flag
`print summary(checks)`	(prints)	Print "TOTAL: N PASS, M FAIL out of K"

Appendix F: The Precision Report Metadata

The `precision_report` function returns a dictionary with complete comparison metadata. This is the measurement instrument underlying `chk_precision`.

Key	Type	Meaning
numeric digits	mpf	Digits of numeric agreement (from digits of). mp.inf if EXACT.
source digits	int	Significant digits in the published string (from count sig digits)
string match	bool	True if rendered string matches published over all source characters
diverge pos	int	Character position of first divergence (−1 if match)
diverge pub	str	Character at divergence in published string (empty if match)
diverge got	str	Character at divergence in rendered string (empty if match)
rendered	str	The Fraction rendered to source digits + 10 significant figures
published	str	The input published string, stripped
exceeds source	bool	True if numeric agreement exceeds source significant digits

Example outputs for different constant types:

A measured constant with full string match:

```
precision_report(alpha_inv, "137.035999177")
# → numeric_digits: inf (EXACT — Fraction IS the published value)
#   source_digits: 12
#   string_match: True
#   diverge_pos: -1
#   exceeds_source: True
```

A very small constant where mpmath renders in scientific notation:

```
precision_report(a_0, "0.00000000005291772105 44")
# → numeric_digits: inf (EXACT)
#   source_digits: 12
#   string_match: False (format divergence at position 0)
#   diverge_pos: 0
#   diverge_pub: "0"
#   diverge_got: "5"  (mpmath rendered "5.29...e-11")
#   exceeds_source: True
```

The string divergence for a_0 is a rendering format mismatch, not a data error. The numeric check passes EXACT. The system correctly reports the divergence without failing the check — this is why pass/fail is numeric and string comparison is diagnostic.

Appendix G: The Q335 Basis Architecture

The Q335 = 2^335 basis is the arithmetic substrate for all transcendental constants in the series. Every irrational or transcendental constant is represented as an integer numerator p divided by the shared denominator 2^335.

Design parameters:

Parameter	Value	Rationale
Bit depth	335	Gives $335 \times \log_{10}(2) = 100.9$ decimal digits
Denominator	2^335 (shared)	Integer addition/subtraction of numerators directly
Numerator digits	100-102 per constant	Sufficient for 100-digit verification against mpmath
Total constants	31	Core basis (22 from MATH-2) + extended (9 from MATH-3)
Rounding	Nearest integer (round half up)	Minimizes representation error
Corruption check	First 20 digits of numerator in comment	Visual inspection of stored value integrity

The 31 basis constants:

ID	Constant	Digits vs mpmath	Computation Method	Terms
G1	π	102.0	Machin arctan series	160
G2	e	101.9	Factorial series	80
G3	ln(2)	101.2	Arctanh series	160
G4	$\sqrt{2}$	103.2	Newton iteration	10
G5	$\sqrt{3}$	101.4	Newton iteration	10
G6	$\sqrt{5}$	103.0	Newton iteration	10
G7	$\sqrt{7}$	102.0	Newton iteration	10
G8	$\varphi = (1+\sqrt{5})/2$	101.4	Newton iteration	10
G9	$\zeta(3)$	101.6	Apéry-type central binomial series	180
G10	$\zeta(5)$	101.2	Borwein eta acceleration	210
G11	π^2	102.4	Product of G1 $\times$ G1	—
G12	$\zeta(2) = \pi^2/6$	101.5	Quotient of G11 / 6	—

ID	Constant	Digits vs mpmath	Computation Method	Terms
G13	$R_2 = \pi /4$	102.0	Quotient of G1 / 4	—
G14	$R_4 = \pi^2/32$	102.4	Quotient of G11 / 32	—
G15	2π	102.0	Product of 2 × G1	—
G16	$\zeta(7)$	101.2	Borwein eta acceleration	210
G17	$\zeta(9)$	102.0	Borwein eta acceleration	210
G18	$Li_4(1/2)$	101.2	Direct series (recomputed, 500 terms)	500
G19	$Li_5(1/2)$	101.2	Direct series	300
G20	$Li_6(1/2)$	101.1	Direct series	300
G21	$Li_7(1/2)$	100.9	Direct series	300
G22	Catalan	101.6	Euler transform	350
G23	e^ π	103.7	Taylor expansion of exp at π	120
G24	ln(3)	102.7	ln(2) + 2 × arctanh(1/5)	160
G25	ln(5)	102.2	2 × ln(2) + 2 × arctanh(1/9)	160
G26	$K(k^2=1/4)$	101.6	${}_2F_1$ hypergeometric	500
G27	$K(k^2=1/2)$	101.6	${}_2F_1$ hypergeometric	500
G28	$K(k^2=3/4)$	102.7	${}_2F_1$ hypergeometric (recomputed, 800 terms)	800
G29	$E(k^2=1/4)$	102.7	${}_2F_1$ hypergeometric	500
G30	$E(k^2=1/2)$	102.3	${}_2F_1$ hypergeometric	500
G31	$E(k^2=3/4)$	101.4	${}_2F_1$ hypergeometric (recomputed, 800 terms)	800

All 31 constants achieve 100+ digits of agreement with mpmath at 120 dps. The three recomputed constants (G18, G28, G31) originally had 99.93, 65.33, and 68.08 digits respectively — insufficient for the 100-digit platform standard. Recomputation with additional series terms brought all three above 101 digits. Both original and recomputed numerators are stored in the library (p_foo and p_foo_full). The Fraction variables use the _full numerators throughout.

Appendix H: The DATA-4 Entry Type System

Every entry in DATA-4 has a type code that determines how it can be used in computation and what kind of verification is appropriate.

Type	Meaning	Count	Verification Method	Example
E	Exact by SI 2019 definition	7	chk exact against literal Fraction	c = 299792458 m/s
M	Measured (CODATA, PDG, FLAG)	47	chk precision against published decimal string	α^{-1} = 137.035999177
A	Analytical (computed to arbitrary precision)	31	chk against mpmath at 100+ digits	π = 3.14159…
D	Derived (Level 1 applied to Level 2)	17	chk exact for Fraction identities, chk for numerical	gap SM = 218/115
G	Staged (identified by theory, bounded, not measured)	6	chk bool for bounds and properties	M VL $\in$ [1.5, 6.0] TeV
K	Koide derived (from mass fits)	8	chk against computed-from-masses values	K(e, μ ,τ) = 0.666661

Type rules for inference:

If you have…	You may…	You may NOT…
Type E value	Use as exact. No uncertainty.	Assign error bars.
Type M value	Use at published precision.	Assume more digits than published.
Type A value	Use at 100+ digits for Q335 constants.	Assume exact (rounding residual exists).
Type D value	Use, but trace provenance to Level 2 inputs.	Treat as independent of its inputs.
Type G value	Use bounds for range checks.	Quote a single value as measured.
Type K value	Use, but note mass-precision dependence.	Claim higher precision than source masses support.

DATA-4 section inventory:

Section	Entries	Types	Content
A	7	E	SI fundamental: c, h, e, k B, N A, $\Delta\nu$ Cs, K cd
B	13	M	CODATA measured: α^{-1}, m e, m μ, m τ, m p, m p/m e, R∞, a_0, a e, a μ, $\sin^2\theta$ W, α s, μ_0
C	6	M	Electroweak: M Z, Γ Z, M W, m t, m H, G F
D	11	M	Quarks + CKM: m u, m d, m s, m c, m b, sin θ_{12}, sin θ_{23}, sin θ_{13}, m c/m s, m b/m c, m u/m d
E	8	M	Nuclear/hadron: m n, m n−m p, m π^+, m π^0, m K^+, m D, m He4, E D
F	1	M	Spectroscopy: H 1S-2S transition (2466061413187018 Hz, 16 sf)

Section	Entries	Types	Content
G	31	A	Q335 analytical: π , e, ln(2), √2-√7, φ, ζ(3,5,7,9), Li_4-Li_7(1/2), Catalan, e^ π , ln(3,5), K and E at k^2=1/4,1/2,3/4
K	8	M/K	Ratios + Koide: m μ /m e, m τ/m e, m τ/m μ , m n/m p, M W/M Z, m H/M Z, m t/M Z, K(e, μ ,τ)
L	6	G	Cabibbo Doublet staged: M VL bounds, θ_{14} estimate, θ_{24}/θ_{34}/δ_1/δ_2 status
N	17	D	GUT/unification: b_1-b_3 SM, Δb_1-Δb_3 VL, b_1'-b_3' modified, gap ratios (SM/VL/MSSM/measured), b ij matrix, Δ values
Total	**146**		

Appendix I: The Platform Test Coverage Map

Every value in phys24_lib.py is tested. This table maps library sections to platform test parts.

Library Section	Test Part	Check Type	Count	What Is Verified
Section A (SI exact)	Part 4	chk exact	7	Each constant equals its defining Fraction

Library Section	Test Part	Check Type	Count	What Is Verified
Section B (CODATA)	Part 3	chk precision	13	Decimal reconstruction at full published digits
Section C (Electroweak)	Part 3	chk precision	6	Decimal reconstruction
Section D (Quarks/CKM)	Part 3	chk precision	11	Decimal reconstruction
Section E (Nuclear/hadron)	Part 3	chk precision	8	Decimal reconstruction
Section F (Spectroscopy)	Part 3	chk precision	1	Decimal reconstruction (16 sf)
Section G (Q335 analytical)	Part 1	chk (vs mpmath)	31	100+ digits agreement with independent computation
Section G (identities)	Part 2	chk + chk exact	6	$\pi^2 = \pi \times \pi$, $\zeta(2) = \pi^2/6$, $2\pi = 2 \times \pi$, $R_2 = \pi/4$, $R_4 = \pi^2/32$, $\varphi = (1+\sqrt{5})/2$
Section K (Ratios/Koide)	Part 3	chk precision	8	Decimal reconstruction
Section K (Koide computed)	Part 8	chk + chk bool	6	K and a^2 from masses, ordering, $a^2 \neq 2$
Section L (CD staged)	Part 11	chk bool + chk exact	8	Quantum numbers, bounds, asymmetry, democracy
Section N (GUT params)	Part 5	chk exact	15	All betas, shifts, gap ratios as exact Fractions
Section N (b ij matrix)	Part 12	chk exact	9	All 9 entries of the two-loop matrix

Library Section	Test Part	Check Type	Count	What Is Verified
Section N (couplings)	Part 9	chk bool + chk	7	Coupling ranges, gap distance, GUT normalization
Mass ratios (cross)	Part 6	chk	6	m p/m e computed vs stored, etc.
Physical relations	Part 7	chk	3	$R\infty$, a_0, μ_0 from SI formulas
Lattice independence	Part 10	chk bool	3	PDG vs FLAG discrepancy > threshold
Total			**148**	

Appendix J: The Demonstration Script Specifications

Each of the 8 demonstration scripts follows the same structure: header, inputs with DATA-4 entry IDs, computation, results, checks block, footer. Each demonstrates one concept.

phys24_gap_ratio.py — The Gap Ratio

Property	Value
Concept	SM gauge couplings do not unify at one loop
Lines	~178
Checks	5/5
Key computation	gap SM = $(b_1-b_2)/(b_2-b_3)$ = 218/115 vs measured 1.358, miss 39.6%
Library imports used	alpha inv, sin2 tW, alpha s, b1 SM, b2 SM, b3 SM, gap SM, gap measured, inv a1, inv a2, inv a3
Session 3 ancestor	sin2 theta w 1.py (checks 1-3 of 9)
Key checks	SM gap = 218/115 EXACT, gap from betas matches library EXACT, measured gap ~1.358 at 4 digits, miss > 30%, GUT normalization at 11 digits

phys24_democracy.py — Generation Democracy and the Boson Problem

Property	Value
Concept	Fermions contribute zero to the gap ratio — it's a boson problem
Lines	~273
Checks	10/10
Key computation	Decompose betas into gauge + Higgs + fermion. Show fermion gap contribution = 0 exactly.
Library imports used	b1 SM, b2 SM, b3 SM, db per gen, casimir gap, gap SM
Session 3 ancestor	sin2 theta w 1.py (democracy section)
Key checks	Per-gen (4/3, 4/3, 4/3) three EXACT, decomposition sums three EXACT, fermion gap num=0 EXACT, fermion gap den=0 EXACT, boson-only gap = full gap EXACT, pure-gauge gap = 2 EXACT

phys24_cabibbo_doublet.py — The Cabibbo Doublet

Property	Value
Concept	Minimal single-multiplet fix: (3,2,1/6) with Dynkin index derivation
Lines	~305
Checks	10/10
Key computation	Δb from Dynkin formulas, gap 38/27, asymmetry 15, M GUT, proton decay scaling
Library imports used	CD SU3, CD SU2, CD Y, db1 VL, db2 VL, db3 VL, b1 mod, b2 mod, b3 mod, gap SM, gap VL, gap MSSM, gap measured, inv a1, inv a2, M Z
Session 3 ancestor	sin2 theta w 1.py (enumeration + CD sections)
Key checks	Δb = (1/15, 1, 1/3) three EXACT, gap = 38/27 EXACT, MSSM = 7/5 EXACT, asymmetry = 15 EXACT, CD distance < 0.05, M GUT in [10^15, 10^16], MSSM M GUT in [10^16, 10^18], lifetime ratio > 10^5

phys24_two_loop.py — Two-Loop Unification

Property	Value
Concept	Two-loop corrections improve CD unification by 66%
Lines	~270
Checks	8/8
Key computation	Display b ij matrix, present verified Δ values from unification test.py
Library imports used	b1 SM, b2 SM, b3 SM, b ij SM, db1 VL, db2 VL, db3 VL, b1 mod, b2 mod, b3 mod, delta 1loop, delta 2loop, twoloop improvement, gap SM, gap VL, gap MSSM
Session 3 ancestor	unification test.py (6/6 checks)
Key checks	b ij[0][0], [1][1], [2][2] EXACT, Δ 1loop ~−1.17 at 2 digits, Δ 2loop ~−0.40 at 2 digits, improvement ~66% at 2 digits, two-loop closer than one-loop,

phys24_koide_status.py — The Koide Status

Property	Value
Concept	C_3 path closed, amplitude is the open problem
Lines	~327
Checks	10/10
Key computation	K and a^2 for 3 sectors, tautology proof (3 params, 3 data), saddle point demonstration
Library imports used	m e, m mu, m tau, m u, m c, m t, m d, m s, m b, K koide, a2 lep, a2 down, a2 up
Session 3 ancestor	data 4.py (Koide checks) + PHYS-23
Key checks	K(leptons) at 6 digits, a^2 for 3 sectors at 3-4 digits, ordering K lep < K down < K up, PHYS-8 identity at 6 digits, residuals sum ~0, saddle $d^2K > 0$ in (1,−1,0), saddle $d^2K < 0$ in (2,−1,−1), $a^2 \neq 2$

phys24_a2_anatomy.py — The A_2 Anatomy

Property	Value
Concept	QED 2-loop coefficient: 87% cancellation between geometry and arithmetic
Lines	~195
Checks	7/7
Key computation	$A_2 = 197/144 + (3/4)\zeta(3) + R_4 \times (8/3 - 16 \ln 2)$
Library imports used	pi2 f, zeta3 f, ln2 f, R4 f
Session 3 ancestor	a 2 decomposition 0.py (7/7 checks)
Key checks	Decomposition sums to A_2 at 30 digits, A_2 ~−0.3285 at 10 digits, A_2 negative, geometric negative, rational+number positive, cancellation > 80%,

phys24_pslq_null.py — Derivation Beats Search

Property	Value
Concept	PSLQ finds known relations, returns null for unknowns. 82/82 null.
Lines	~221
Checks	4/4
Key computation	Sanity: PSLQ finds $\pi^2 = 6\zeta(2)$. Test: j_{11} independent of 20-constant basis at 100 digits.
Library imports used	(uses mpmath directly for PSLQ and Bessel zeros, library for infrastructure)
Session 3 ancestor	bessel pslq 0.py (6/6 checks)
Key checks	Sanity finds [1, 0, −6], Bessel test returns NULL, j_{11} value at 30 digits, Airy constant at 8 digits

phys24_data4_check.py — The Database

Property	Value
Concept	DATA-4 is self-consistent. 146 entries, representative cross-checks.
Lines	~272
Checks	8/8
Key computation	One check from each of 6 groups + lattice independence + staged entries

Property	Value
Library imports used	m p, m e, mp me, R2 f, alpha inv, m e, c, h planck, e charge, R inf, m mu, m tau, K koide, b1 SM, b2 SM, b3 SM, gap SM, m c, m s, mc ms, M VL lo, M VL hi, CD Y
Session 3 ancestor	data 4.py (38/38 checks)
Key checks	m p/m e at 11 digits, $R_2 = \pi/4$ at 100 digits, R∞ from formula at 11 digits, K(e, μ ,τ) at 6 digits, c = 299792458 EXACT, gap SM = 218/115 EXACT, lattice disc > 10%, CD mass window staged

---

Appendix K: The Script Standard Rule Index

Complete index of all 22 sections in phys24_script_rules.md with one-line summaries.

Section	Title	One-Line Summary
1	Purpose	Scripts are the source of truth. Papers report what scripts produce.
2	Language and Compatibility	Python 3.8+. No walrus, match/case, or assert. Use % formatting.
3	Arithmetic Rule	Fraction in, mpf at display boundary only, f2m for conversion.
4	Precision Rule	100 dps standing, 120 headroom, save/restore, 11 sf minimum, mpf(“string”) thresholds.
5	The phys24 lib Rule	One import. All constants from library. Platform versions by import line.
6	Standard Helpers	10 helpers defined in library. No script redefines them. chk precision pass/fail is numeric.
7	Check Block Format	checks = [], standard block at end, print summary(checks), unique tags.

Section	Title	One-Line Summary
8	Output Structure	Header → Inputs → Computation → Results → Checks → Footer.
9	Naming	phys24 prefix for Session 4 scripts.
10	Scope	One concept per script. 50-100 lines target. 150 max.
11	DATA-4 References	By library variable name with entry ID. State full usage explicitly.
12	Cross-Verification Rule	Every key output compared to Session 3 scripts. Disagreement stops work.
13	What Scripts Do Not Do	No files, no math, no float, no assert, no raw prints, no hardcoded constants.
14	Scripts In Chat	Written in chat as code blocks. Human runs. One script per prompt.
15	Error Recovery	FAILs: diagnose. Genuine discrepancy: stop all work. Never paper over. Hard crashes are diagnostic.
16	The math Ban	No import math. Use mpmath for log, sqrt, pi. Fraction handles gcd.
17	Determinism	No time, random, locale, network, filesystem, environment, or platform dependence.
18	Units	Every label includes units or “dimensionless.”
19	Output Medium Rules	Scripts in chat, papers as markdown, diagrams as Python, no JS/widgets/polls.
20	Directory Structure	paper name/code/ for scripts and library, paper name/supplementary/ for rules.

Section	Title	One-Line Summary
21	The Docstring Standard	Title, one-line description, Backed by with check count, Platform with test counts.
22	Summary	Four categories: computation rules, platform rules, workflow rules, check rules.

Appendix L: The Level 1 / Level 2 Enforcement in Code

The platform enforces the series' epistemological rule (PHYS-21) through code, not just documentation. This appendix catalogs every place where the distinction is made explicit in phys24_lib.py.

Constants classified by type in library comments:

Every constant has its DATA-4 entry ID and type in the comment:

```
alpha_inv = Fraction(137035999177, 10**9)      # B1: alpha^-1, 12 source di

b1_SM = Fraction(41, 10)                        # N1: U(1)_Y, GUT normaliza

M_VL_lo = Fraction(1500000, 1)                  # L1: 1.5 TeV lower bound (
```

Type M values (measured) carry source digit counts. Type D values (derived) are computed from other library values in situ. Type G values (staged) carry bound annotations.

The Koide enforcement:

```
# These are MEASURED values, not the hypothesis.

# a^2 = 2 is the Level 1 hypothesis. a^2 = 1.9999630688 is the

# Level 2 measurement. The library stores the measurement.

a2_lep = Fraction(19999630688, 10**10)    # K7: NOT exactly 2
```

Self-test check:

```
chk_bool("a^2(leptons) is NOT exactly 2",

     a2_lep != Fraction(2, 1),
```

```
    "a2_lep = %s (Level 2 measurement, not Level 1 hypothesis)" %
    mp.nstr(f2m(a2_lep), 11), checks)
```

The Cabibbo Doublet staged enforcement:

Level 1 properties are exact Fractions (computed, not measured):

```
CD_SU3 = 3                      # color triplet — Level 1, exact
CD_Y = Fraction(1, 6)           # hypercharge — Level 1, exact
```

Level 2 properties are bounds, not values:

```
M_VL_lo = Fraction(1500000, 1)     # Level 2, staged lower bound
M_VL_hi = Fraction(6000000, 1)     # Level 2, staged upper bound
theta14_est = Fraction(45, 1000)   # Level 2, staged estimate
```

The platform test checks bounds, not values:

```
chk_bool("M_VL lower bound = 1.5 TeV",
    M_VL_lo == Fraction(1500000, 1), ...)
```

The gap ratio Level distinction:

Level 1 (exact Fraction from group theory):

```
gap_SM = (b1_SM - b2_SM) / (b2_SM - b3_SM)     # = 218/115 exactly
gap_VL = (b1_mod - b2_mod) / (b2_mod - b3_mod) # = 38/27 exactly
```

Derived (Level 1 formula applied to Level 2 inputs):

```
gap_measured = (inv_a1 - inv_a2) / (inv_a2 - inv_a3)   # ~1.358
```

The gap_measured value depends on three Level 2 measurements (α^{-1}, $\sin^2\theta_W$, α_s). If any of those measurements improve, gap_measured changes. The Level 1 gap ratios (218/115, 38/27) never change — they are properties of the gauge group and particle content.

Appendices D-L: Extended supporting tables for DISC-12. Every table traces to a verified artifact or documented design decision. Published April 2, 2026.

DISC-12 is thorough and well-structured. The central argument holds, the tables are correct, and the appendices provide genuine reference value. Here are the errata and annotations.

Errata

E2: Verification count inconsistency. The abstract and Section 4 cite "367 checks across 17 components." Appendix C totals 329 across 16 components and notes the discrepancy: "the 367 total includes the DATA-4 38/38 counted separately from data_4.py." This is confusing. DATA-4's 38/38 IS data_4.py — they are the same artifact. The non-overlapping count is 329. Use 329 consistently throughout, or if 367 is intentional (counting the DATA-4 paper verification separately from the script), explain the accounting in the abstract, not just in a footnote in Appendix C.

E3: Table 6 row "Unused import `euler as mgamma`" — source attribution. The table says "External reviewer" caught this. For the record: this was caught by the review Claude during the script audit, not by an external party. The three-party workflow is writing Claude, human operator, review Claude. "External reviewer" is misleading in this context. Change to "Review Claude script audit."

E4: Appendix G, $Li_5(1/2)$ computation method. The table says $Li_5(1/2)$ was computed with "300" series terms. The library shows `p_li5_full = p_li5` (no recomputation needed — the original achieved 101.2 digits). But $Li_4(1/2)$ needed recomputation from 300 to 500 terms. The table correctly shows Li_4 at 500 terms. However, it lists Li_5 through Li_7 at 300 terms each, which is the original count, not a recomputation. This is correct but could be misread as "these were also recomputed." Add a note: "Li_5, Li_6, Li_7 did not require recomputation — original 300-term series already achieved 100+ digits."

E5: Appendix F, a_0 example. The example shows `diverge_got: "5"` with the note "(mpmath rendered '5.29…e-11')". This is correct behavior — mpmath renders small numbers in scientific notation while the published string uses plain decimal. The text explains this correctly. However, the example should note that `chk_precision` was specifically designed to handle this: pass/fail is numeric, string comparison is diagnostic. This is already stated in Section 5 but the appendix example should cross-reference it: "See Section 5: pass/fail is numeric; string divergence is diagnostic."

Annotations

A1: Section 1 — the a^2_lep example is perfectly placed. This is the concrete motivating case that makes the problem real instead of hypothetical. The progression from "a future session reads '$a^2 \approx 2$' and writes Fraction(2,1)" to "only a platform that stores 1.9999630688 and tests it is NOT exactly 2 can enforce the distinction" is the strongest paragraph in the paper. No change needed.

A2: Section 6 — the three-party workflow is a genuine methodological contribution. The separation of authoring (writing Claude), execution (human), and review (review Claude) such that no single party can introduce and hide an error is not standard practice in LLM-assisted work. Most LLM workflows have the LLM write and evaluate its own output. The HOWL workflow forces external validation at every step. This deserves the prominence it gets.

A3: Section 7 — the _full convention defense is correctly framed. "Insurance purchased with bytes and redeemed with hours" is the right way to explain a design decision that looks wasteful now but prevents refactoring later. The three constants that already demonstrate the value (Li_4, K(3/4), E(3/4)) are the concrete proof. No change needed.

A4: Section 9 — the Dynkin formula debugging arc. The paper describes the cabibbo_doublet.py development but doesn't give the specific iteration count. From the session record: iteration 1 had kappa=2 giving 2 $\times$ the library values (3 FAIL), iteration 2 removed kappa but used (2/3) for SU(3) instead of (1/3) giving 1 FAIL on Δb_3, iteration 3 used the correct coefficients (2/5, 2/3, 1/3) giving 10/10 PASS. The review Claude then caught the float() on a fourth pass. Consider adding the iteration details to Table 6 or Section 9 — they show the check system catching progressively subtler errors.

A5: Appendix D — cycle counts are valuable data. The table showing phys24_lib.py took 5 cycles while most demo scripts took 1 cycle is an empirical finding about where complexity concentrates in platform development. The foundation artifacts need the most iterations because errors there propagate everywhere. This pattern would likely replicate in other platform builds. Worth highlighting in Section 9 or 10 as a general observation.

A6: Appendix H — the type inference rules table is operationally useful. The "If you have Type M, you may NOT assume more digits than published" rule prevents a specific class of error that occurs when a value is known to 5 digits but someone uses it in a 100-digit computation and trusts all 100 output digits. This table should be referenced from PHYS-24 Section 12 (The Database) in a future errata if the series continues.

A7: Appendix J — script line counts. The table gives line counts (178, 273, 305, etc.) that are higher than the "50-100 lines target" in the script standard Section 10. This is because the line counts include the standard boilerplate (docstring, imports, section headers, check block, summary). The computational content of each script is within the target. The script standard should clarify: "50-100 lines of computation, not counting boilerplate" — or the line counts in Appendix J should note which portion is computation vs boilerplate.

[**HOWL-DISC-12-2026**] Paternal Operationalism. Github: https://github.com/ghowland/papers/tree/main/papers/DISC/H
DISC-12-2026

[**HOWL-DISC-6-2026**] Paternal Operationalism. Github: https://github.com/ghowland/papers/tree/main/papers/DISC/HO
DISC-6-2026

[**HOWL-DISC-7-2026**] Paternal Operationalism. Github: https://github.com/ghowland/papers/tree/main/papers/DISC/HO

DISC-7-2026

[**HOWL-DISC-8-2026**] Paternal Operationalism. Github: https://github.com/ghowland/papers/tree/main/papers/DISC/HO
DISC-8-2026

[**HOWL-DISC-11-2026**] Paternal Operationalism. Github: https://github.com/ghowland/papers/tree/main/papers/DISC/H
DISC-11-2026